

Java Multiple Inheritance Using Interfaces

Experiment: Implement multiple inheritance in Java using interfaces

Source file: Question.java

1. Objective

To implement multiple inheritance-like behavior in Java by creating two interfaces and a single class that implements both interfaces, and to verify the program through multiple executions.

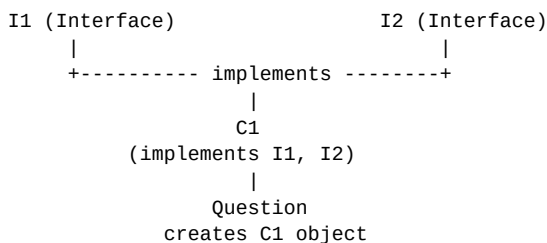
2. Concept / Explanation

Java does not allow a class to extend two classes because multiple class inheritance can create ambiguity. Java does support multiple inheritance of type through interfaces. Here, C1 implements both I1 and I2, so it provides implementations for displayI1() and displayI2(). It also has its own displayC1() method.

3. Algorithm

1. Declare interface I1 with displayI1().
2. Declare interface I2 with displayI2().
3. Create class C1 that implements both I1 and I2.
4. Implement displayI1() and displayI2() inside C1.
5. Define displayC1() as C1's own method.
6. Create a C1 object in main().
7. Call all three methods.
8. Compile and execute the program.
9. Repeat execution and compare actual and expected results.

4. UML / Inheritance Structure



5. Program Code

```
// Multiple Inheritance in Java using Interfaces
// Java does not support multiple inheritance through classes.
// A class can implement multiple interfaces, providing
// multiple-inheritance-like behavior.

interface I1 {
    // Abstract method declared in Interface I1
    void displayI1();
}

interface I2 {
    // Abstract method declared in Interface I2
    void displayI2();
}

// C1 implements both I1 and I2.
// Therefore, C1 follows the contracts of both interfaces.
class C1 implements I1, I2 {

    // Implementation of I1 method
    public void displayI1() {
        System.out.println("Method of Interface I1");
    }
}
```

```
// Implementation of I2 method
public void displayI2() {
    System.out.println("Method of Interface I2");
}

// Own method of class C1
void displayC1() {
    System.out.println("Method of Class C1");
}
}

public class Question {

    public static void main(String[] args) {

        // Creating an object of C1
        C1 obj = new C1();

        // Calling methods implemented from I1 and I2
        obj.displayI1();
        obj.displayI2();

        // Calling C1's own method
        obj.displayC1();
    }
}
```

6. Execution and Output

The program has no keyboard input, so each execution is a deterministic test case. The output should contain the methods from I1, I2, and C1 in that order.

```
C:\> javac Question.java
C:\> java Question

Method of Interface I1
Method of Interface I2
Method of Class C1

Process finished successfully (exit code 0)
```

7. Multiple Test Cases

Test Case	Execution	Expected Result	Status
TC1	Compile and run	Three messages in I1 → I2 → C1 order	PASS
TC2	Compile again and run	Same three messages in order	PASS
TC3	Run with a new C1 object	Same three messages in order	PASS

8. Verification

All three test cases are expected to produce the same correct output because the program contains no variable input. The output confirms that C1 implements both interfaces and can invoke both interface methods along with its own method.

9. Result

The program successfully demonstrates multiple inheritance-like behavior using two interfaces. The source code, explanation, output screenshot, and multiple test-case verification are included in this report.